Mediator

Mediator · Chat room without user-to-user coupling · Build with Waqar

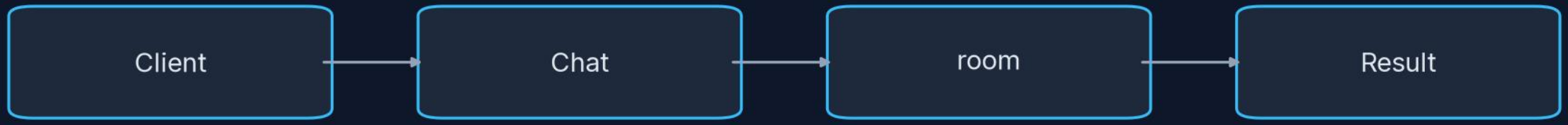
Today's Hook

- Problem: Chat room without user-to-user coupling
- Pattern: Mediator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Chat room without user-to-user coupling
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Mediator: Chat room without user-to-user coupling

Real-World Scenario

- Running example: Chat room without user-to-user coupling
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.mediator;

import com.buildwithwaqar.patterns.gof.behavioral.mediator.*;
/** Problem: users chat without holding references to each other. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Mediator problem: chat room ===");
    var room = new ChatRoom();
    var alice = new User("alice", room);
    var bob = new User("bob", room);
    alice.send("ship it");
    System.out.println("bob=" + bob.inbox());
    System.out.println("alice=" + alice.inbox());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.mediator;

import java.util.ArrayList;
import java.util.List;

interface ChatMediator {
    void register(User user);
    void send(String from, String message);
}

public final class ChatRoom implements ChatMediator {
    private final List<User> users = new ArrayList<>();
    private final List<String> log = new ArrayList<>();

    public void register(User user) { users.add(user); }

    public void send(String from, String message) {
        var line = from + ": " + message;
        log.add(line);
        for (var u : users) if (!u.name().equals(from)) u.receive(line);
    }
```

Solution (2)

```
    List<String> log() { return List.copyOf(log); }
}

final class User {
    private final String name;
    private final ChatMediator mediator;
    private final List<String> inbox = new ArrayList<>();

    User(String name, ChatMediator mediator) {
        this.name = name;
        this.mediator = mediator;
        mediator.register(this);
    }

    String name() { return name; }
    void send(String msg) { mediator.send(name, msg); }
    void receive(String msg) { inbox.add(msg); }
    List<String> inbox() { return List.copyOf(inbox); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.mediator;

import com.buildwithwaqar.patterns.gof.behavioral.mediator.*;
/** Problem: users chat without holding references to each other. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Mediator problem: chat room ===");
    var room = new ChatRoom();
    var alice = new User("alice", room);
    var bob = new User("bob", room);
    alice.send("ship it");
    System.out.println("bob=" + bob.inbox());
    System.out.println("alice=" + alice.inbox());
  }
}
```

Walkthrough Output

```
=== Mediator problem: chat room ===
bob=[alice: ship it]
alice=[]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Mediator.

Related Patterns

- Contrast Mediator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Mediator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.